

Architecture & Infrastructure

Polycarpic — a virtual Single Family Office

Owner: architect agent **Status:** Draft v1 — pending SecEng / DevOps / QA folds and Principal review

Traces to: PRD v1 (Approved 2026-09-27), kickoff record **Last updated:** 2026-09-27

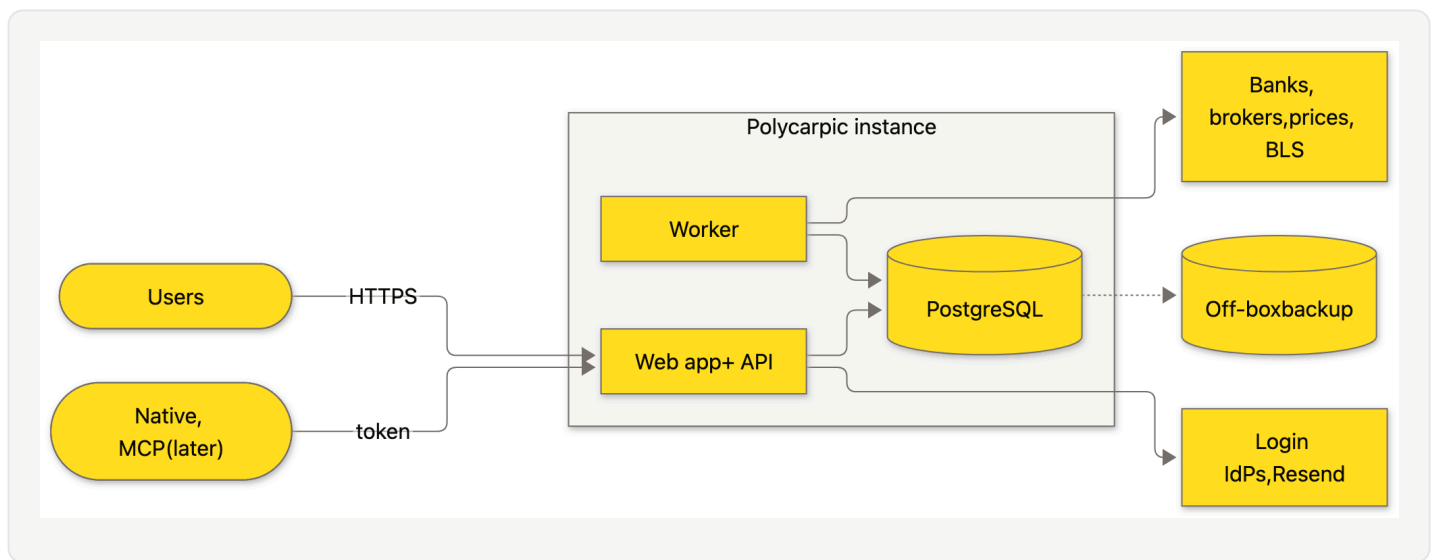
CONTENTS

- 1. System Context
- 2. Components & Layer Model
- 3. Data Model & Database Invariants
- 4. Data Flow
- 5. Tech Stack & Rationale
- 6. Plan-phase Research: GL Build-vs-Adopt, Migration Tool
- 7. Deployment Topology
- 8. CI/CD Pipeline
- 9. Integration Points
- 10. Trade-offs & Alternatives
- 11. Milestone Map
- 12. Open Questions

1. System Context

The system and the actors around it. C4 context level.

Polycarpic is a single self-hosted web application backed by one PostgreSQL database. Users (the Principal, household members holding a membership grant, later native iOS/macOS clients and a user-scoped MCP server) reach it over HTTPS. Inside, a genuine double-entry general ledger lives in Postgres under row-level security keyed on the *membership* join. Outbound, the system pulls transactions from bank/brokerage aggregators through a provider-agnostic adapter interface (a deterministic fake first, then SimpleFIN, Plaid, and direct broker APIs), end-of-day prices and the CPI-U series as reference data, and sends transactional email through Resend. Identity is local (BetterAuth in our own Postgres) with Google / Apple / GitHub as OAuth identity providers. Nothing about a user's books leaves the box except through a provider call the user initiated or an owner-initiated export.

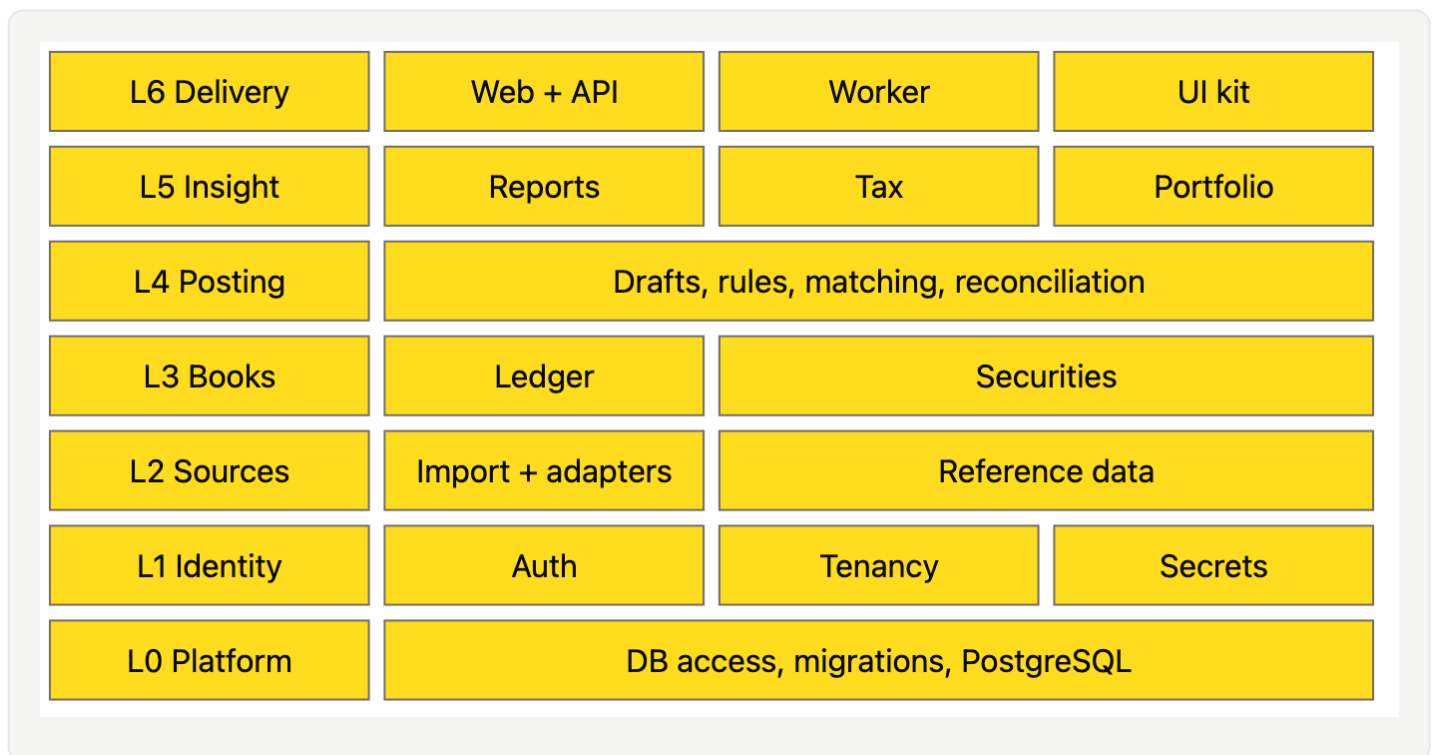


2. Components & Layer Model

Layers depend only downward. Each layer is one or more workspace packages with its own tests; the boundary is enforced by lint, not convention.

2.1 Layer model

Kickoff §3 asks for layers defined before details, depending only downward, with narrow interfaces. The model below is the contract. **Rule:** a package may import only packages in a strictly lower layer or in its own layer's shared `contracts` ; `packages/*` never import `SvelteKit`; only `packages/db` opens connections, and only through `withTenant()` / `withSystem()` (see §3.3). Enforced by `eslint-plugin-boundaries` (or `dependency-cruiser`) in CI.



Why import sits below the books. Kickoff §2.6 calls import "the lowest non-infrastructure layer". That holds for the *raw* side — adapters, the raw-transaction store and de-dup depend on nothing domain-level, so they sit at L2. Confirmation, rules and transfer matching need both raw rows and the ledger, so they are split into `posting` (L4). This keeps "raw data arrives first" true while dependencies stay strictly downward. The ledger never imports the importer: a journal entry carries an opaque `source_ref` ; the two-way link and the posted-status flip happen inside one database transaction owned by `posting` .

2.2 Components

Component	Responsibility	Owner agent	Exposes	Consumes
packages/db	Owns the schema, migrations, Postgres roles, and the only path to a connection — a tenant- or system-scoped transaction.	backend-lead	<code>withTenant(userId, fn)</code> , <code>withSystem(job, fn)</code> , Drizzle table objects, migration runner	PostgreSQL
packages/auth	Configures BetterAuth (sessions, OAuth, MFA, bearer tokens) against our Postgres; maps a request to a user id.	backend-lead	<code>getSession(request)</code> , auth route handler	db, Resend, OAuth IdPs
packages/tenancy	Entities, entity types, parent links, memberships, and the owner/manager/viewer permission matrix as data.	backend-lead	entity/membership service, <code>can(role, action)</code>	db
packages/secrets	Seals and opens aggregator credentials with HPKE (X25519 / HKDF-SHA256 / AES-256-GCM; SECURITY §6.3). <code>web</code> only seals, under the public key; only <code>worker</code> holds the private keyring and can open.	backend-lead (SecEng sign-off)	<code>seal(plain, aad) → {enc, ciphertext, key_id, alg_version}</code> (web); <code>open(sealed, aad)</code> (worker only, enforced by lint)	public keys (web); keyring file (worker)
packages/import + adapters	Provider-agnostic adapter interface; fetches, normalizes, hashes and stores raw transactions and reported balances.	backend-lead	<code>ImportAdapter</code> interface, <code>ingest(connectionId, range)</code>	db, secrets, provider APIs
packages/refdata	Global reference series — EOD prices, CPI-U,	backend-lead	<code>PriceProvider</code> , <code>SeriesProvider</code> ,	db, BLS, price

Component	Responsibility	Owner agent	Exposes	Consumes
	per-jurisdiction per-year tax tables — behind provider interfaces.		lookups	provider
packages/ledger	Chart of accounts and journal entries; posting and reversal; balances as of a date. DB enforces balance and immutability.	backend-lead	postEntry() , reverseEntry() , balances(asOf)	db, tenancy
packages/securities	Lots, lot matching, quantities, book value adjustments (return of capital, amortization), holdings as of a date.	backend-lead	lot service, holdings(asOf)	db, ledger, refdata
packages/posting	Turns raw rows into drafts, applies rules, matches transfers, confirms drafts into entries, reconciles custodial balances.	backend-lead	draft queue, confirm(draftId) , rule engine, matcher, reconciliation	import, ledger, securities
packages/reports	Evaluates declarative report definitions (rows by account range / tag / type, formula lines, period or entity columns).	backend-lead	run(definition, params) → ReportTable	ledger, securities, portfolio
packages/tax	Running annualized accrual, safe-harbor derivation, period-cutoff freezes, Form 2210 / Schedule AI export.	backend-lead	estimate(entity, jurisdiction, asOf) , freeze(period)	ledger, refdata, tenancy
packages/portfolio	NAV, allocation vs. target, inflation	backend-lead	nav() , allocation() ,	securities, refdata

Component	Responsibility	Owner agent	Exposes	Consumes
	adjustment of any time series.		<code>deflate(series, base)</code>	
<code>packages/ui</code>	Owned shadcn-svelte components, LayerChart wrappers, design tokens imported from <code>docs/DESIGN/</code> .	frontend-lead	Svelte components, token CSS (and a Swift token export later)	DESIGN tokens
<code>apps/web</code>	SvelteKit pages (server <code>load</code> calls packages directly) and <code>/api/v1</code> thin route wrappers with Zod schemas → OpenAPI.	frontend-lead (pages) / backend-lead (routes)	HTML, <code>/api/v1</code> , <code>openapi.json</code>	every L1–L5 package, ui
<code>apps/worker</code>	Schedules the idempotent job units (import poll, EOD prices, CPI fetch, tax period freeze; Cash-in-Transit aging is computed on read, not a job).	backend-lead / devops-engineer	none (outbound only)	job units in L2–L5

2.3 Interface contracts worth pinning now

- **ImportAdapter** — `id`; `capabilities` (transactions, balances, holdings, trades); `connect(input) → sealedCredential`; `listAccounts(cred)`; `fetchTransactions(cred, account, range) → RawTxn[]`; `fetchBalances(cred, account) → ReportedBalance[]`; `revoke(cred)` (provider-side revocation, PRD §5). The fake source implements exactly this and is registered as `fake`; switching an instance to SimpleFIN is configuration. A CSV/OFX file adapter implements the same interface with no stored credential (SecEng's preferred credential-free path).
- **RawTxn** — provider id, provider account id, posted date, amount (decimal string), currency, payee/memo, provider transaction id (nullable), provider payload (JSON). Before storing the payload, the adapter redacts full account numbers, PANs and any credential fields (SECURITY A-9). The de-dup hash is computed by `import`, never by the adapter: `sha256(provider, provider_account_id, provider_txn_id ?? (date, amount, normalized payee, per-day ordinal))`, enforced by a unique index.

- **PriceProvider / SeriesProvider** — `fetch(symbolOrSeries, range) → Observation[]` with a `source` and a priority; CPI-U reuses the price-series pattern (PRD §5). Fake EOD provider: seeded random walk per ticker.
- **Job unit** — `run(ctx, params): Promise<Result>`; must be idempotent, short (< 60 s per invocation, chunked otherwise), and triggerable by either scheduler (§7).
- **Screen DTOs** — each page's server `load` returns a DTO type exported by the owning package. The same types back `/api/v1`, so the web UI and native clients share one contract. The list of screens comes from DESIGN (ux-designer); DTOs get pinned per screen in each package's design file (kickoff §2.13).

2.4 Screen → DTO ownership

Screens and their data come from DESIGN § Screen data contract. A screen that spans packages gets a composite DTO assembled in `apps/web` from package DTOs. A panel whose package hasn't shipped yet renders an empty state, so the Dashboard ships in POLY-M3 and gains the tax and allocation panels as POLY-M4 and POLY-1.0 land.

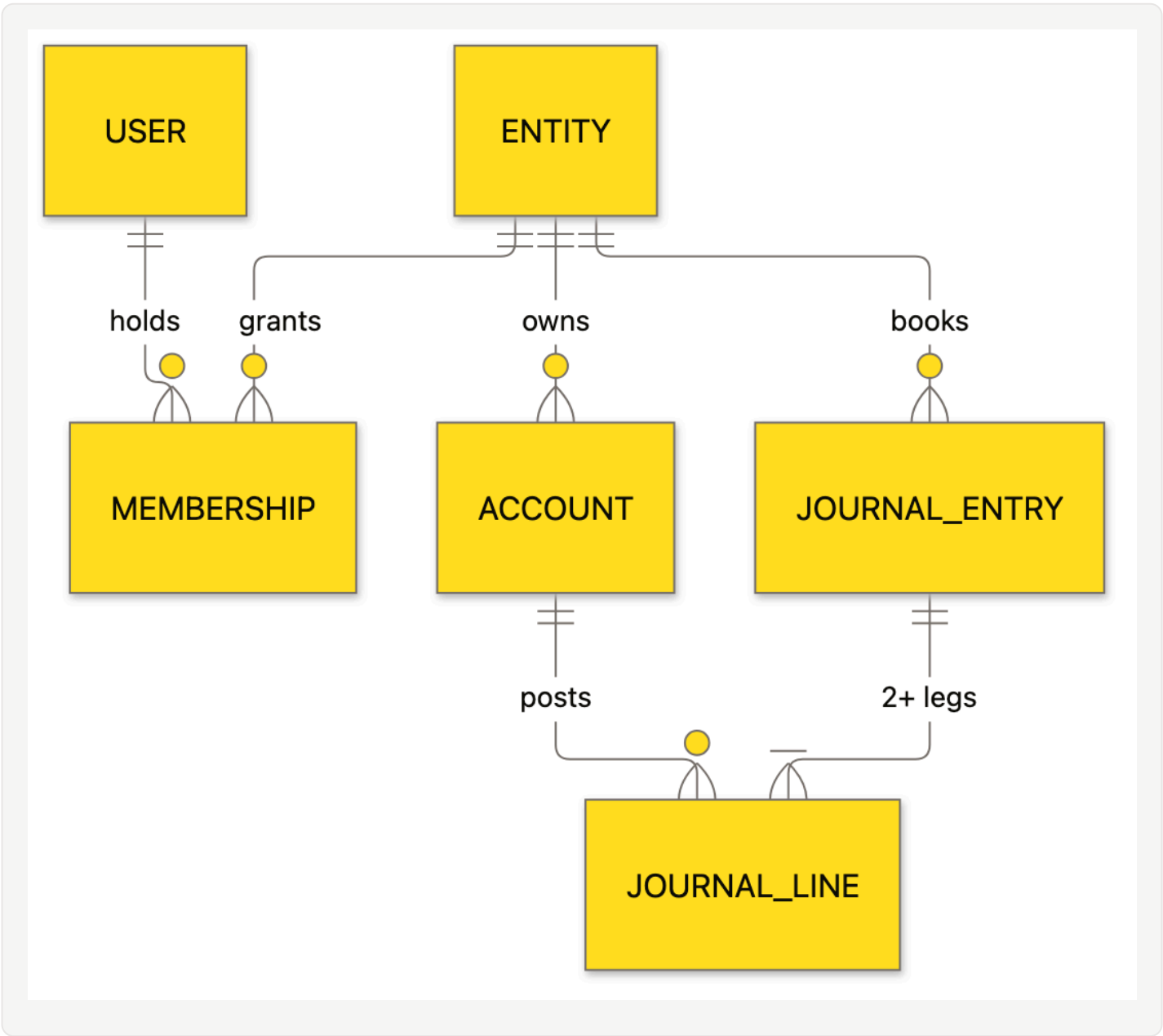
Screen	DTO(s) → owning package	First milestone
Dashboard	<code>NetWorthSeries</code> , <code>CashFlowByPeriod</code> , <code>RecentActivity</code> → ledger / posting; <code>TaxPosition</code> → tax; <code>AllocationVsTarget</code> → portfolio	POLY-M3 (panels fill in by M4 and 1.0)
Ledger	<code>DraftList</code> , <code>CitExceptions</code> → posting; <code>PostedEntryList</code> (with reversal links and actor) → ledger	POLY-M2 (CIT exceptions in M3)
Accounts	<code>BalanceSheet</code> (GL / custodial) and <code>NavDelta</code> → reports; <code>ReconciliationStatus</code> → posting	POLY-M3 status, M5 statements
Entities	<code>EntityList</code> , <code>MembershipList</code> , invite command → tenancy	POLY-M1
Securities	<code>LotTable</code> → securities; <code>AllocationVsTarget</code> , <code>BookVsMarketSeries</code> → portfolio	POLY-M4
Import	<code>ConnectionList</code> , <code>ImportBatch</code> (progress; new / duplicate / needs-category counts) → import + posting	POLY-M2

Model consequence: batch review needs an `import_batch` row per ingest run, holding status and counts. Rows skipped by `ON CONFLICT DO NOTHING` are counted into the batch as duplicates instead of vanishing silently (§3.1).

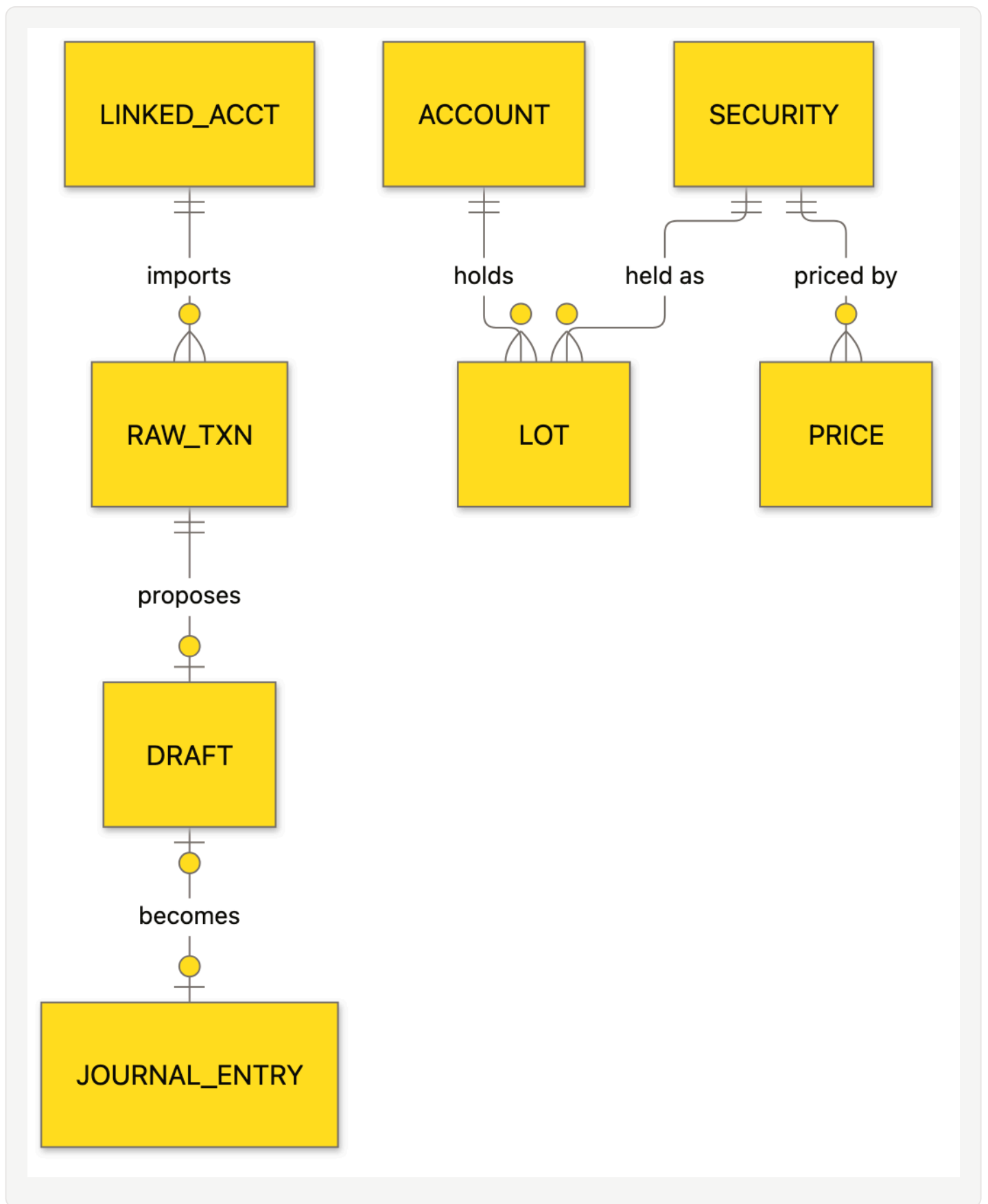
3. Data Model & Database Invariants

Logical model at ARCH grain; column-level design lives in each package's design file.

3.1 Core entities



Tenancy and the ledger, above. Import and securities, below. The table that follows lists every table group.



Keys: every tenant-visible primary key is a UUID, never a sequential integer, so ids leak neither counts nor ordering across tenants (SECURITY A-6). Tenancy columns (`entity_id`) and actor columns are not updatable.

Table group	Tenant key	Mutability	Notes
<code>user</code> (BetterAuth)	self	mutable; deletion tombstones	Account deletion turns the user row into a tombstone ("deleted user") and removes credentials, sessions and memberships. The row itself is never deleted, and entries are never UPDATED. <code>actor_id</code> FKs are <code>ON DELETE NO ACTION</code> , and BetterAuth's <code>deleteUser</code> is overridden (SECURITY A-8).
<code>entity</code> , <code>membership</code> , <code>entity_type</code>	entity id; types: <code>owner_user_id</code> or null (system)	mutable; tenancy columns not updatable	Setting a parent link requires owner on both the child and the parent (SECURITY A-11). Tax-classification attribute on <code>entity</code> (individual; trust grantor / non-grantor; business pass-through / C-corp). User-defined types readable by creator plus holders of any membership on an entity of that type — the narrow cross-tenant metadata exception SecEng flagged.
<code>account</code>	entity id	mutable until first posting; then name/parent only	Number, name, parent, class (asset / liability / equity / income / expense), normal side, currency, <code>is_custodial</code> + custodian, default tax-treatment tag. Seeded per entity: Cash in Transit, Suspense, Opening Balance Equity, prepaid-tax assets per jurisdiction.

Table group	Tenant key	Mutability	Notes
journal_entry , journal_line	entity id (denormalized onto the line)	append-only (trigger)	Entry: posting date (exactly one), description, actor_kind (user / rule / system) + actor_id , reverses_entry_id , source_ref . Line: account, currency , native_amount , base_amount , fx_rate , category, tax-treatment tag, memo. Amounts are numeric , signed (debit > 0); never JS floats.
import_batch , raw_transaction , reported_balance	entity id (via linked account, denormalized)	raw rows insert-only; status draft → posted (then frozen by trigger); batch mutable while running	Unique de-dup hash. Full provider payload kept for audit; payload is readable by manager+ only, never by viewers (SECURITY §4.2.1). Batch holds run status and new / duplicate / needs-category counts (§2.4).
draft , rule , transfer_match	entity id	mutable until confirmed	Draft carries the proposed entry (lines as JSON) and an actor. Rule carries auto_post opt-in (default false).
provider_connection	entity id + linking user	mutable	Sealed credential only: enc , ciphertext , key_id , alg_version , with AAD = (connection, entity, provider, key_id, alg_version) (SECURITY §6.3). poly_app may INSERT and UPDATE those columns but never SELECT them; only poly_worker reads them. linked_by_user_id NOT NULL is the principal that job writes run under. A principal_invalid state is set when that user's membership is revoked or their account is deleted; it surfaces on the dashboard and writes an audit_event , so it never fails

Table group	Tenant key	Mutability	Notes
			silently (SECURITY §4.4). On unlink, <code>web</code> enqueues a revoke job; <code>worker</code> opens the credential, revokes at the provider, then deletes the row.
<code>security</code> , <code>price_observation</code> , <code>cpi_observation</code> , <code>tax_table</code>	global reference data	writable by system jobs / operator import only	Price: unique (security, date, source) + source priority (borrowed from mosko-fintech). Tax table: jurisdiction × filing year → due dates, cutoffs, multipliers, cumulative percentages — a new year is a data row.
<code>lot</code> , <code>lot_match</code> , <code>lot_adjustment</code>	entity id	append-only	Lot-match is an append-only many-to-many junction with a <code>match_seq</code> ; the latest batch is current (borrowed from mosko-fintech 032).
<code>tax_period_freeze</code>	entity id	append-only	Annualized income, tax, cumulative required, paid — per jurisdiction per period; source of the 2210 / Schedule AI export.
<code>report_definition</code>	owner user id (clones) / null (system templates)	system templates read-only; clones mutable	Upgrades replace system templates in place; clones are never touched.
<code>attachment</code> , <code>attachment_blob</code> , <code>entry_attachment</code>	entity id	attachment insert-only; links deletable by manager+ (audited)	Supporting documents such as statements and receipts, stored as <code>bytea</code> in Postgres (see §3.5). They are RLS-covered, backed up, exported and purged along with everything else.
<code>audit_event</code>	entity id (nullable after purge)	append-only, outside purge cascade	Records purges, exports, revocations, membership changes, and <code>principal_invalid</code> transitions; survives the purge it describes.

Table group	Tenant key	Mutability	Notes
			Readable by owners only (SECURITY §4.2.1).

3.2 Invariants the database enforces (not the app)

1. **Balanced entries.** A DEFERRABLE INITIALLY DEFERRED constraint trigger on `journal_line` checks $\sum \text{base_amount} = 0$ per entry at commit, and at least two lines. The app computes balance too, for good error messages; the trigger is the backstop.
2. **Append-only posted rows.** BEFORE UPDATE OR DELETE triggers on `journal_entry`, `journal_line`, `lot`, `lot_match`, `tax_period_freeze` and on `raw_transaction` where `status = 'posted'` raise, and so do BEFORE TRUNCATE statement triggers on the same tables. No login role holds TRUNCATE. The one exception is DELETE by `current_user = 'poly_purge'` for the entity named in `app.purge_entity` (§3.4). Corrections go through `reverseEntry()`: a mirror entry referencing the original, plus a new entry.
3. **Two-way link.** Confirming a draft inserts the entry and flips the raw row to `posted` with the entry id in one transaction. A check ensures a posted raw row always has an entry id and that the entry's `source_ref` points back.
4. **Single posting date.** One date column per entry; a `posting_date_allowed(entity, date)` function hook (always true in v1) is the seam where period locking plugs in later without a schema change.
5. **Same-entity legs.** Every line's account belongs to the entry's entity (composite FK (`entity_id`, `account_id`)). Cross-entity transfers are two entries, one per entity's books (see OQ-6).
6. **Actor recorded.** `actor_kind + actor_id` are NOT NULL on entries and drafts, and are *forced* by a BEFORE INSERT trigger from the transaction's context rather than merely defaulted. `actor_kind = 'user'` requires `actor_id = app.current_user_id()`, so a caller cannot attribute an entry to someone else (SECURITY A-5).

3.3 RLS mechanism and connection-pooling safety

Confirmed by SecEng in SECURITY §4.3 / §12 (OQ-2), with the four additions listed last. The permission matrix is SECURITY §4.2.1: roles are uniform across entity types.

- **Roles.** `poly_owner` (NOLOGIN; owns every object); `poly_migrator` (LOGIN; used only by the migrate step; runs migrations as `set role poly_owner ... reset role`, which mosko-fintech measured as the working shape); `poly_app` (LOGIN; DML grants only; NOBYPASSRLS; not the table owner, so RLS applies); `poly_worker` (LOGIN; NOBYPASSRLS; reference-data writes plus tenant DML limited to the import path, SECURITY §4.4); `poly_authz` (NOLOGIN; owns the authorization functions and nothing else); `poly_purge` (NOLOGIN; NOBYPASSRLS; owns the purge function;

DELETE on purgeable tables only). Role hygiene follows SECURITY §4.3(k). The operator's superuser bypasses RLS by definition — documented as an accepted risk (PRD §6).

- **Context per transaction, never per session.** `withTenant(userId, fn)` opens a transaction and runs `select set_config('app.user_id', $1, true) ( is_local = true )` before any query. The setting dies at COMMIT/ROLLBACK, so a pooled connection cannot carry a previous request's identity. This works with the in-process `node-postgres` pool on the VPS and with Neon's transaction-mode pooler. Session-level `SET` is banned by lint.
- **Default deny.** Policies read context only through `app.current_user_id() = nullif(current_setting('app.user_id', true), '')::uuid`. The `nullif` matters: once a backend has set the GUC and the transaction has ended, `current_setting` returns `''`, not `NULL`. Without `nullif`, a reused backend would compare against an empty string instead of denying cleanly (SECURITY A-3). Every tenant table has RLS `ENABLE d` and `FORCE d`, including drafts, raw rows, lots and staging tables.
- **Policy shape.** `using (entity_id = any (app.visible_entities()))` for reads; writes add `app.has_role(entity_id, 'manager')` or `'owner'` per the permission matrix. Both functions are `STABLE SECURITY DEFINER`, owned by `poly_authz` (not `poly_owner`, because under `FORCE` RLS the owner is itself subject to policies). They read `membership` through a role-targeted `SELECT ... TO poly_authz` policy, with `search_path = ''`. Every definer function is on a CI-checked allowlist (SECURITY A-1, §4.3(h)). Parent/child structure is never consulted — structure grants nothing.
- **System jobs.** `withSystem(job, target, fn)` sets `app.user_id` to the target's principal and `app.actor = system:<job>`. It never runs context-free; if no principal resolves, the job errors. Targets come from one allowlisted definer function, `app.job_targets(job)`, which returns `(entity_id, principal_user_id, connection_id)` and is executable only by `poly_worker`. Entity-bound jobs with no connection (the tax-period freeze) run as the entity's earliest owner membership. Reference-data policies are role-targeted (`TO poly_worker`), and `poly_app` gets `SELECT` only. Cash-in-Transit aging is computed on read by the exceptions report, not stored by a job (SECURITY A-12). Tenant writes by a job (import polling) run under the linking user's membership (`provider_connection.linked_by_user_id`) with `actor = system:<job>`, so authority and attribution stay separate. There is no system bypass clause in tenant policies (SECURITY §4.4, OQ-3).
- **SecEng additions (OQ-2).** (a) The RLS coverage probe is derived from `pg_catalog`, never from a hand-maintained list. (b) `poly_app` also loses `CREATE` on `public`. (c) Policies are per-command with an explicit `WITH CHECK` on every write; `FOR ALL` is banned. (d) The inversion canary stays in the battery. The review added (e) to (l) (SECURITY §4.3):
 - (e) Authorization functions are owned by `poly_authz`.
 - (f) Context is read in the `nullif` form.
 - (g) `TRUNCATE` triggers.
 - (h) `search_path = ''`, plus a definer allowlist.
 - (i) Tenancy columns are not updatable.

- (j) Views are `security_invoker = true`, and there are no materialized views over tenant data (a matview has no RLS). A report needing a cache materializes into a tenant table under RLS.
- (k) Role hygiene.
- (l) UUID keys.
- **Proof.** Every migration run in CI executes an RLS battery: an *inversion self-test* (an unprotected canary table must leak, proving the probe has teeth — borrowed from mosko-fintech), a per-table cross-tenant probe generated from the catalog (a table with RLS disabled fails the build), and a pool-reuse test. That test must run on a backend that has *already* served a request: tenant A's transaction, then tenant B on the same physical connection sees zero A rows, and a context-free query on that dirty backend sees zero rows. The full battery list is SECURITY §12.3 (qa-engineer).

3.4 The privileged purge path

A sole owner's whole-entity purge (PRD §5) runs through one `SECURITY DEFINER` function, `app.purge_entity(entity_id, export_id, reauth_at)`. The function is owned by the dedicated NOLOGIN, NOBYPASSRLS role `poly_purge`, with `search_path = ''`, and `EXECUTE` is granted to `poly_app` (SECURITY §4.5, OQ-4).

Why not `poly_owner`: under FORCE RLS the owner is itself subject to policies. Append-only tables have no DELETE policy, so an owner-run purge would delete zero rows and still report success (T-51).

How the deletes are allowed: each purgeable table carries `CREATE POLICY purge ... FOR DELETE TO poly_purge USING (entity_id = nullif(current_setting('app.purge_entity', true), ''))::uuid)`. The append-only triggers allow DELETE only when `current_user = 'poly_purge'` and the row's entity matches. `poly_app` can set the GUC but can never become `poly_purge`.

The guards run in this order, in one transaction:

1. Re-authentication within 5 minutes. The route checks it, and the function requires the timestamp as an argument.
2. Sole-owner check against `app.current_user_id()`.
3. An `export.completed` audit event for this entity, by this user, within 24 hours, carrying a SHA-256.
4. No tenant write to the entity since the export's snapshot marker. The entity row is locked `FOR UPDATE`.
5. An `entity.purged` `audit_event` is written before the first delete. `audit_event` has no FK to `entity` and no purge policy.
6. Every connection has been revoked at the provider by the worker revoke job. A failed revocation blocks the purge.
7. Set `app.purge_entity` transaction-locally and delete in dependency order, attachments included.
8. Assert zero rows remain for the entity in every purgeable table before commit (fail loud).

Not permitted: purge by a job; purge through the API without typed confirmation of the entity name; purge of a partial range; purge while a second owner exists. **Rejected:** `DISABLE TRIGGER` (DDL, affects every tenant), tombstoning (breaks PRD §5), `BYPASSRLS` (broader than needed). An optional cancellable 24-hour delay is pending the Principal (SECURITY §12.3 Q2).

3.5 Attachments

PRD §5 includes attachments (statements, receipts, closing documents) in export and purge, but v1 had no storage design (SECURITY A-14). The design:

- **Location:** Postgres `bytea` in an entity-scoped `attachment` table. It is RLS-covered like every other tenant table, and it automatically falls inside backup, export and purge. It works unchanged on the secondary Vercel + Neon target.
 - *Rejected for v1:* a filesystem or S3-compatible object store. That would be a second access-control system to keep in lockstep with RLS, and a second thing to back up, export and purge.
 - *Cost:* database size. At household scale, a few hundred MB per year is expected (unmeasured).
 - *Revisit* if the attachment table exceeds a size set in INFRA.
- **Limits:** 10 MB per file. Type allowlist: PDF, PNG, JPEG, and CSV/OFX (kept as import evidence). The type is sniffed server-side, and the client-declared type is never trusted. A SHA-256 is stored for de-dup and for the export manifest.
- **Serving:** only through an app route inside `withTenant`, with `Content-Disposition: attachment`, `X-Content-Type-Options: nosniff`, the sniffed `Content-Type`, and a restrictive CSP. Nothing is rendered inline.
- **Linking:** `entry_attachment` links an attachment to a journal entry, draft or account. A link doesn't change the entry, so attaching to a posted entry is allowed. Unlinking is `manager+` and writes an `audit_event`. Attachment bytes themselves are insert-only.
- **Boundary:** uploads cross TB-11 (untrusted file ↔ parser). Attachments are stored, never parsed. CSV/OFX import files are parsed by the import adapter under TB-11's controls.

SecEng conditions (approved on security, all binding):

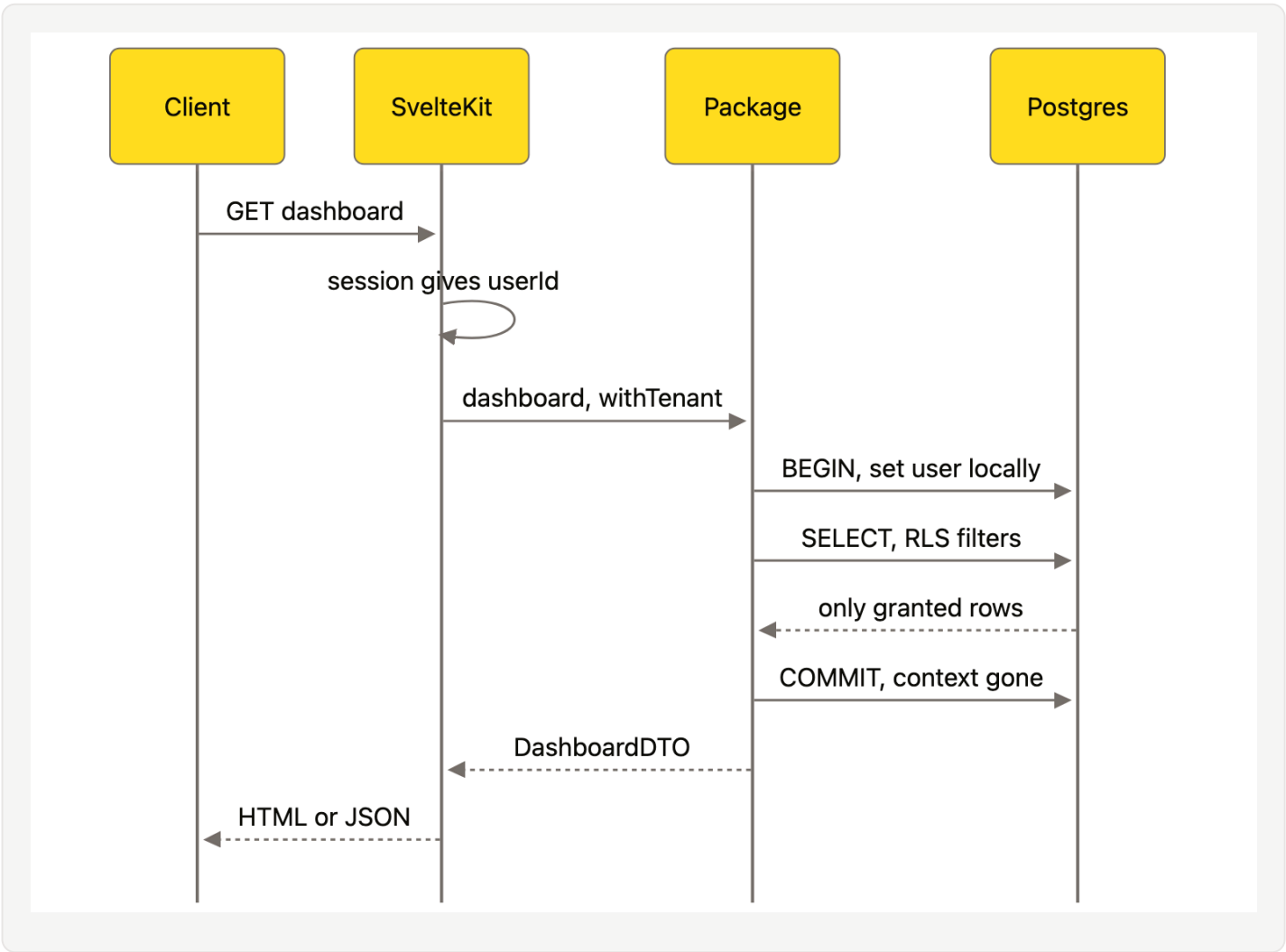
1. *Filename:* `Content-Disposition` uses the RFC 6266 `filename*=UTF-8''` form, with CR/LF and quotes stripped. The CSP is `default-src 'none'; sandbox`.
2. *Entity binding:* `attachment` and `entry_attachment` both carry `entity_id`, with composite FKs, so a link can never cross entities. Viewers read; `manager+` uploads.
3. *Purge:* both tables get the `poly_purge DELETE` policy and the append-only trigger exception, and are included in the zero-rows-remain assertion (C-50).
4. *Quota:* a per-entity storage quota and an upload rate limit protect the disk. The numbers are set in INFRA.

5. *Isolation*: the bytes live in their own table (`attachment_blob` , 1:1 with `attachment`), so list and DTO queries never select `bytea` . Content is never parsed, and no AV scanning is an accepted consequence of that.

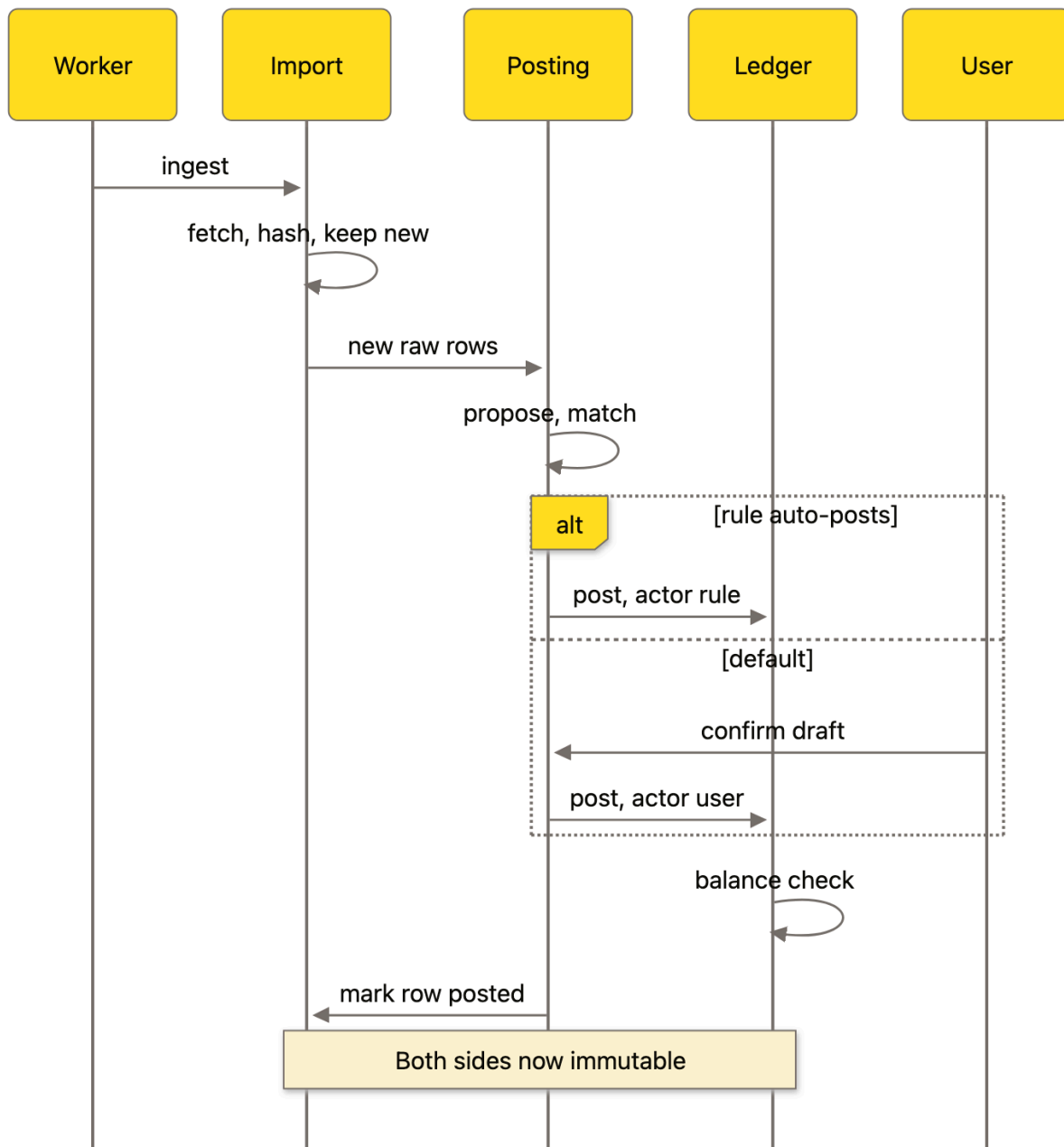
4. Data Flow

Only flows that surface a non-trivial design choice.

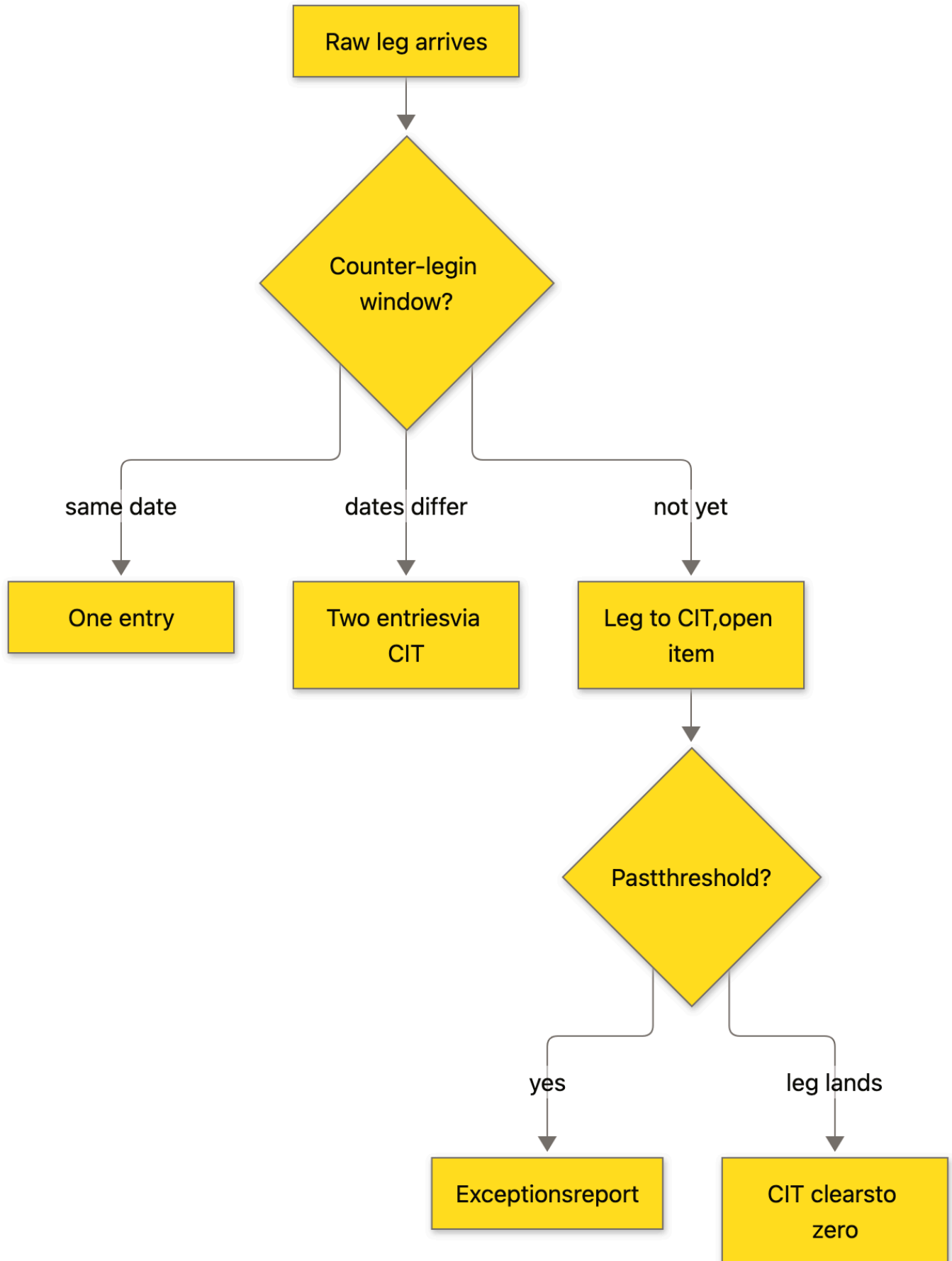
4.1 Any authenticated read (PRD Story 1, 8)



4.2 Import → draft → confirm (PRD Story 2, 3)

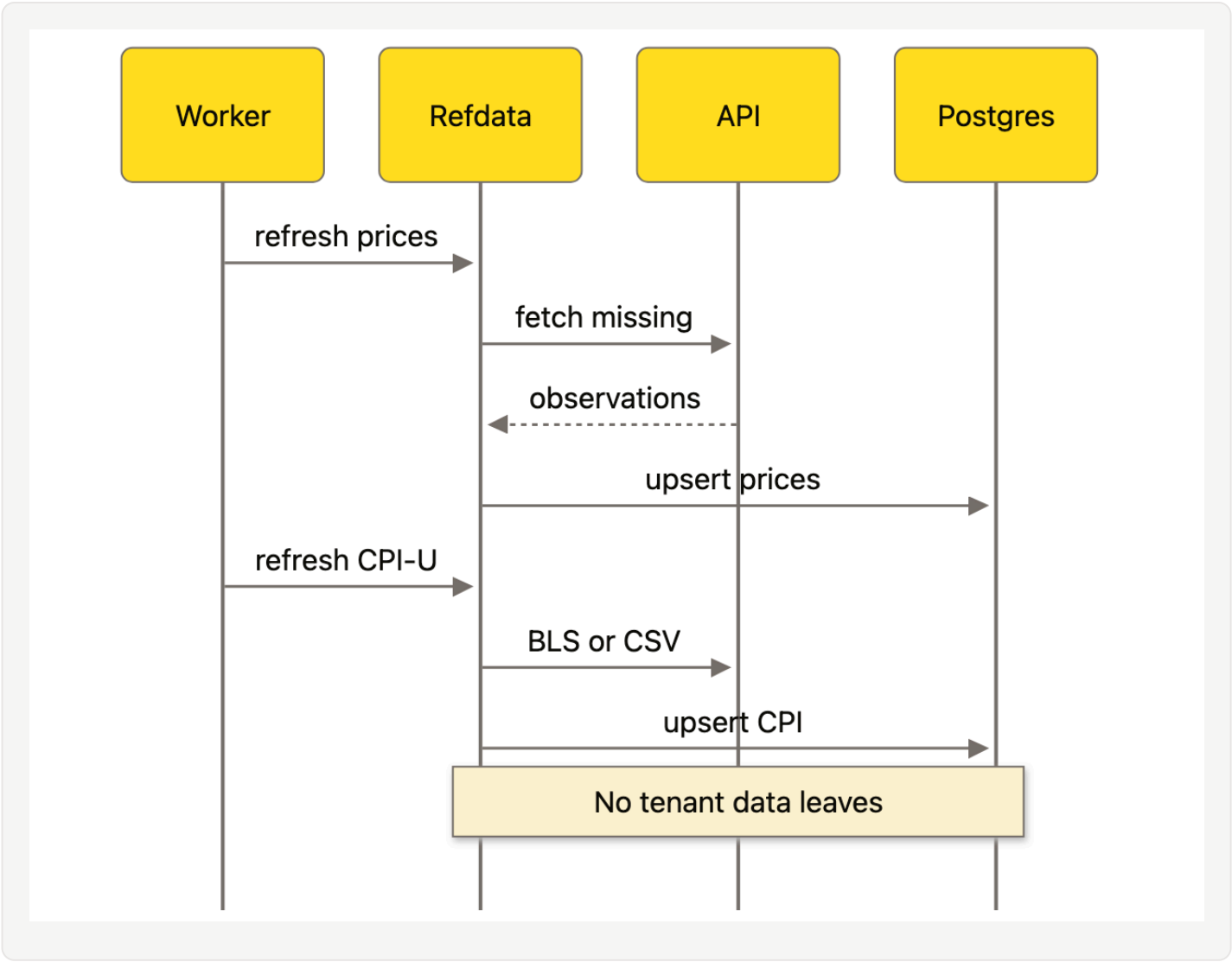


4.3 Transfer matching and Cash in Transit (PRD §5)



Matching lives in `posting` (the import side of posting), never inside `ledger.postEntry()`. Cash-in-Transit aging is computed on read by the exceptions report, not by a job (SECURITY §4.4).

4.4 Reference data: EOD prices and CPI-U (PRD Stories 5, 9)



4.5 Estimated-tax accrual (PRD Story 4)

No sequence diagram. `tax.estimate(entity, jurisdiction, asOf)` is a pure function over (a) year-to-date income by tax-treatment tag from the ledger, (b) the tax table row for that jurisdiction and filing year, (c) prior-year filed tax (an entity attribute per year), and (d) the prepaid-tax account balance. It returns pending liability, both safe-harbor values, the required payment and the gap. A job unit calls `freeze()` the day after each period cutoff and appends a `tax_period_freeze` row. The estimate is recomputed on read, never stored, except the frozen period figures.

5. Tech Stack & Rationale

Justify every row. "We know it" is a valid reason — write it down.

Layer	Choice	Why	Alternatives considered
Language	TypeScript (strict) end to end, Node LTS	One language across UI, API, jobs and schema; the Principal's prior project (mosko-fintech) is TS/SvelteKit.	Go / Python backend (a second language for no gain at this scale)
Frontend framework	SvelteKit + shadcn-svelte + Tailwind; LayerChart	Kickoff §2.9: owned component source, headless primitives; shadcn-svelte charts are LayerChart. Monarch/Origin feel from dashboard-01.	Flowbite Svelte (rejected: fixed look, ApexCharts)
API	SvelteKit <code>/api/v1</code> routes, Zod schemas, OpenAPI generated and committed (CI fails if stale)	Thin wrappers over packages; one contract for native clients and a future MCP server.	Separate API server (tRPC / Fastify): a second deployable for no gain
Data store	PostgreSQL, latest GA major (18 at Plan time; bump deliberately)	RLS, deferred constraint triggers, <code>numeric</code> , <code>SECURITY DEFINER</code> — the whole integrity story lives here.	SQLite (no RLS), external ledger services (excluded by kickoff §2.5)
ORM / query layer	Drizzle ORM	TS schema as the single source; RLS policies and roles declared in schema; BetterAuth ships a Drizzle adapter, so auth tables share one migration chain. See §6.2.	Kysely + generated types; Prisma (weak RLS / raw-SQL story, heavy engine)
Migrations	drizzle-kit <code>generate</code> (SQL files, committed) + hand-written custom SQL migrations for triggers / functions / roles,	Plain SQL in the repo, a tracking table, never hand-run (kickoff §2.13). Conditional on the M1 spike — see §6.2.	dbmate (runner-up), Atlas, graphile-migrate, Supabase CLI (mosko-fintech incumbent)

Layer	Choice	Why	Alternatives considered
	applied by a one-off migrate container		
General ledger	Home-grown schema in our Postgres (build)	See §6.1.	pgledger (adopt), mosko-fintech's journal model
Money arithmetic	Postgres <code>numeric</code> ; decimal strings on the wire; a decimal library (e.g. <code>big.js</code>) in TS	No binary floats anywhere near money.	Integer minor units (awkward for share quantities and FX rates)
Auth	BetterAuth (Drizzle adapter), OAuth: Google / Apple / GitHub; MFA; bearer plugin; email via Resend	Kickoff §2.8: identity in our DB, no leased tier.	Clerk (rejected: external identity store), Lucia (maintenance mode)
Jobs	Plain idempotent job units; VPS: <code>apps/worker</code> with a Postgres-backed scheduler (pg-boss); Vercel: Vercel Cron → an authenticated internal route that runs the same unit	No Redis, no extra service; Postgres is already the system of record. Dual-target rule (kickoff §2.8).	BullMQ + Redis (extra service), system cron in the container (no retries or visibility)
Monorepo	pnpm workspaces + Turborepo (affected-only)	Kickoff §2.10; per-package tests and versions.	Nx (heavier), plain pnpm filters (no caching)
Tests	Vitest per package; Postgres via a disposable container (Testcontainers or CI service); pgTAP for the RLS / trigger battery; Playwright smoke suite (login, import a batch, confirm a draft, view a statement)	Kickoff §2.13. DB invariants are tested in SQL where they live.	Jest (slower ESM story)
Hosting / runtime	Primary: Hetzner VPS, Docker Compose (Caddy, web, worker, Postgres). Secondary: Vercel + Neon, CI-verified	Kickoff §2.8: owns the box; adapter-only difference.	Fly.io, Railway (leased), Kubernetes (overkill)

Layer	Choice	Why	Alternatives considered
Reverse proxy / TLS	Caddy	Automatic HTTPS in one short config file, which suits the novice-operator runbook.	Traefik, nginx + certbot
Observability	Structured JSON logs (pino) with a redaction list (credentials, tokens, account numbers); Postgres slow-query log; uptime ping. OTel export is optional	Cheap stack. Logs must be safe by construction because PII lives in memos.	Hosted APM (subscription)

6. Plan-phase Research

The two research items the PRD (§10) assigns to the architect. Each ends in a decision candidate for `process/DECISIONS.md`.

6.1 General ledger: build vs. adopt

Scope (kickoff §2.5): the ledger lives in our Postgres under our RLS. External services (Formance, TigerBeetle, Blnk), plain-text engines and ERPs are already out. The question is only: a home-grown schema, or a library that manages tables inside our own Postgres.

Candidate	What it is	Fit against our requirements
pgledger (pgross, MIT)	A single SQL file of tables, functions and views: accounts with denormalized balance + version, two-legged transfers, entries carrying previous/current balance.	✗ No tenant column, no RLS. ✗ The primitive is a two-legged transfer; our entries are multi-leg with one posting date. Batched transfers approximate this, but reports would have to re-group them. ✗ Single currency per account and no <code>native / base / fx_rate</code> per line. ✗ No hierarchical CoA, normal side, custodial flag, or reversal link. ✗ No versioned migrations yet (its own TODO). Upgrades would mean diffing a vendored file.
TS/Node double-entry libraries (e.g. Medici)	App-level ledger libraries.	✗ Medici is MongoDB. We found no mature TS library that manages Postgres tables under host-app RLS (web survey, 2026-09-27).
mosko-fintech journal model	Single-entry <code>account_trans</code> legs grouped post-hoc by <code>journal</code> ; GL derived by an imputation function (<code>fn_gl_entries</code>); <code>users_id</code> as sole tenant anchor.	✗ Not a stored double-entry GL, and direct-owner tenancy contradicts our membership model. Its <i>patterns</i> port well, though (below).
Home-grown	~6 tables (§3.1) + 2 triggers + a posting service.	✓ Fits every requirement directly. Cost: we own correctness. We mitigate that with DB-enforced invariants (§3.2), property-based tests (random balanced / unbalanced entries), and the RLS battery.

Recommendation: build. A GL core is small. The hard parts here are membership tenancy under RLS, multi-leg / multi-currency-ready lines, the securities sub-ledger, and declarative reports, and no

candidate library provides any of them. Adopting pgledger would mean forking it to add all of them, which is building with extra steps.

Borrow from pgledger: append-only entries as the source of truth; optional per-line running balance (deferred until a measured need — at household scale, `SUM` over an indexed `(entity_id, account_id, posting_date)` is expected to be fast (unmeasured; the dashboard <2 s NFR is the guard)).

Borrow from mosko-fintech: the immutability-trigger fence (004), the append-only lot-match junction with `match_seq` (032), price observations with source priority (019), structural de-dup by unique index + `ON CONFLICT DO NOTHING` (021), the RLS inversion self-test (tests/00), account-number masking (002), and the NOLOGIN-owner / `set role ... reset role` migration ownership pattern. **Leave behind:** direct-owner `users_id` tenancy, post-hoc journal grouping, and Supabase-specific `auth.uid()` / JWT policy clauses.

6.2 Migration tool

Constraints (kickoff §2.13): migrations are files committed with the code and never hand-run; CI applies them to a fresh Postgres on every run; the VPS runs a one-off migrate step before the app starts; Vercel runs it as a build step against Neon. On top of that, our schema is heavy on triggers, SECURITY DEFINER functions, roles and RLS policies.

Incumbent check (mosko-fintech): 120 hand-written plain-SQL files applied with `supabase db push`, tracked in `supabase_migrations.schema_migrations`. Lessons worth keeping: a tracking table is non-negotiable (a bare `psql -f` loop was ruled out there); migrations must be tested *through the real apply path*, not `psql` (their `set local role` was a measured no-op under `db push`); objects need a NOLOGIN owner distinct from the applying login. The tool itself doesn't carry over: Supabase CLI assumes a Supabase stack we don't run.

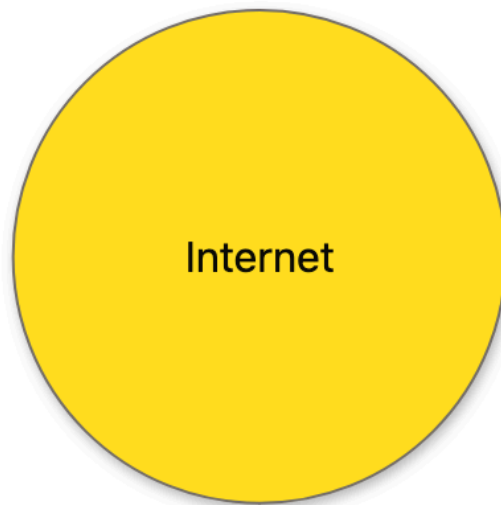
Tool	For	Against
Drizzle Kit (recommended)	TS schema → generated SQL; <code>pgPolicy</code> / <code>pgRole</code> / <code>enableRLS</code> in schema; custom SQL migrations in the same chain for triggers and functions; BetterAuth's Drizzle adapter keeps auth tables in one chain; the query layer and the migration tool share one schema, so types never drift.	Triggers and functions aren't modelled, so they live in hand-written files. Policy diffing maturity is unmeasured — hence the spike.
dbmate	Plain SQL, tiny, language-agnostic, boring; tracking table; trivially run in a container.	No schema-as-code. Types would come from a codegen step (Kysely), and BetterAuth tables go through its own CLI generator.

Tool	For	Against
Atlas	Declarative diffing, strong linting.	Some Postgres object types sit in the paid tier (unmeasured); a second schema language (HCL) alongside TS.
graphile-migrate	Excellent SQL-first dev loop (<code>current.sql</code>).	Postgres-only SQL workflow with no TS schema; smaller community.

Recommendation: Drizzle Kit, gated by an M1 spike. Spike acceptance: (a) a `pgPolicy` round-trips through `generate` twice with no spurious diff; (b) BetterAuth's generated Drizzle schema migrates cleanly; (c) a custom SQL migration containing a trigger and a SECURITY DEFINER function applies through the real runner as `poly_migrator` with objects owned by `poly_owner` , and the tracking row is written; (d) the same runner works against Neon from a Vercel build step; (e) FORCE RLS, a role-targeted `pgPolicy` (`to:`), and a definer function owned by a non-owner NOLOGIN role (`poly_authz` / `poly_purge`) all round-trip through `generate` and apply as `poly_migrator` (SECURITY A-13). If (a), (c) or (e) fails, fall back to **dbmate** + **Kysely codegen**. That fallback changes only `packages/db` , since every other package consumes `withTenant()` and typed table objects.

7. Deployment Topology

*Environments, network paths, trust boundaries. DevOps owns the INFRA-level detail; cross-reference **SECURITY**.*



Hetzner VPS

Caddy, TLS

staging: fake data

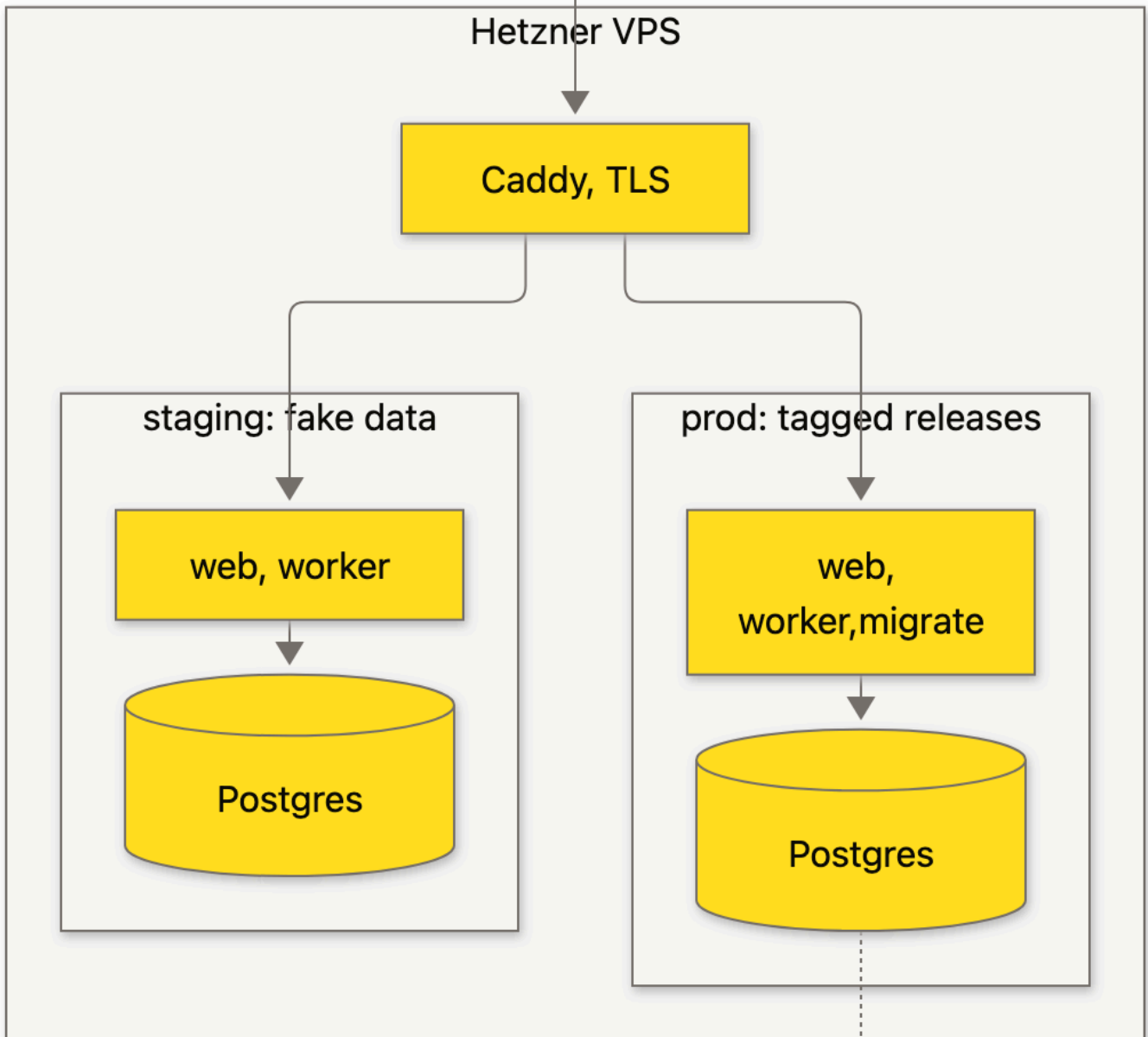
web, worker

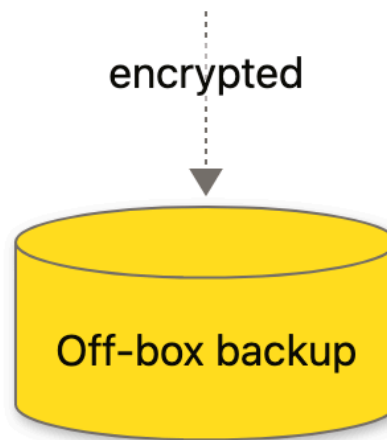
Postgres

prod: tagged releases

web,
worker,migrate

Postgres





- **Trust boundaries** (numbering and controls from SECURITY §3):
 - TB-1: internet ↔ Caddy.
 - TB-2: clients ↔ app.
 - TB-3: app / worker ↔ Postgres (limited roles, RLS).
 - TB-4: `web` (seals with public keys only) ↔ `worker` (holds the private keyring, mounted into worker only; plaintext exists only inside one `open()` call frame). Redefined by SECURITY §6.3.
 - TB-5: worker ↔ providers (egress only).
 - TB-6: app ↔ OAuth IdPs and Resend.
 - TB-7: box ↔ backup store (encrypted before it leaves the box).
 - TB-8: operator SSH ↔ box (superuser; accepted risk R-1).
 - TB-9: ordinary write path ↔ purge path.
 - TB-10: prod ↔ staging compose projects on one host (shared kernel; accepted risk R-4).
 - TB-11: user upload (CSV / OFX, attachments) ↔ web parser.
 - TB-12: supply chain (GitHub CI / Actions / Renovate / registry) ↔ prod images. Whoever can publish an image that prod runs owns the box.
- **Postgres is never exposed publicly.** It lives on the compose network only; the operator reaches it through `docker compose exec` or an SSH tunnel.
- **Only Caddy publishes ports** (SECURITY C-70): every other service (`web` , `worker` , Postgres, migrate) has no `ports:` mapping at all, reachable only over the compose network. Prod and staging run as separate compose projects on separate compose networks — no shared network, so a staging container can't reach a prod one by DNS name even if TB-10's shared kernel is compromised. No Docker socket is ever mounted into a container. Every container runs non-root, `read_only` , with `cap_drop: [ALL]` and `no-new-privileges` ; a compose-policy lint enforces this in CI (C-70).
- **The firewall is the Hetzner Cloud Firewall**, not host `ufw` (SECURITY C-71): it sits outside the host, so a Docker-published port can't bypass it the way it bypasses `ufw` . Only 22 (SSH, key-only, no root login), 80 and 443 are open. Unattended security upgrades are enabled on the box.

- **Environments:** *dev* — local compose (Postgres + web + worker) with the `fake` provider; *CI* — fresh Postgres service per run; *staging* — same VPS, separate compose project and database, fake data only, auto-deploys every merge to `main`; *prod* — tagged releases only (kickoff §2.13). Staging on the same box is a cost trade-off (§10).
- **Secondary target:** the same app built with `adapter-vercel` against Neon, as **two Vercel projects from the same repo** (resolves OQ-13; SECURITY C-61 — see §7.6). CI builds it and runs the smoke suite against a Neon branch (or a local Postgres standing in for one); no Vercel-specific APIs anywhere.
- **Sizing:** CX32 (4 vCPU / 8 GB RAM) as the starting VPS — see §7.3.

7.1 Environments

Environment	Data	Deploy trigger	Access
dev	<code>fake</code> provider only	local compose, manual	developer machine only
CI	fresh Postgres service per run, torn down after	every PR	GitHub-hosted runner only
staging	fake data only (§10)	auto, every merge to <code>main</code>	same VPS, separate compose project + DB; reachable behind Caddy basic auth / IP allowlist — not indexed, not linked from prod
prod	real data	tagged releases only, via <code>/merge-pr</code>	public, TLS via Caddy

7.2 Backup & Disaster Recovery

Resolves OQ-9.

- **Staging co-location:** staging stays on the prod VPS (confirms §10's trade-off). Staging holds fake data only, so its compromise has no ledger-confidentiality blast radius; splitting to a second box roughly doubles the fixed monthly cost for a single-household deployment with no compliance driver forcing separation. Revisit only if the app is offered to other self-hosters at meaningfully larger scale, or staging starts touching anything beyond fake data.
- **Backup store:** Hetzner Storage Box (same account, cheap) as the default, configured for **append-only** access (a restricted SSH/SFTP subaccount with no delete/overwrite permission on existing objects — Hetzner Storage Box's "no substitution" / read-append mode (**unmeasured**) until the C-

73 cannot-delete test in the restore drill below actually passes against it); an S3-compatible bucket with Object Lock in compliance mode is the documented drop-in alternative for self-hosters outside Hetzner. Either way the store credential itself cannot delete or overwrite history (SECURITY C-73's own test: "a test that the store credential cannot delete").

- **Mechanism, revised per SECURITY §6.6 (asymmetric recipient, not restic's symmetric repository password):** a nightly `pg_dump --format=custom` logical backup and continuously-shipped WAL segments are each encrypted with `age` (X25519, the same primitive family as the credential keyring's HPKE scheme) to a **recipient public key**, then uploaded. The box holds only the `age` *public* key — sufficient to encrypt, useless to decrypt. This is a deliberate change from an earlier `restic`-based draft: `restic`'s repository password is symmetric, so a box holding it to *write* backups could also *read* them, which fails SECURITY §6.6's "box holds only the public half" requirement outright. Dedup/incremental snapshotting is given up for this; at single-household data volumes the cost is negligible.
- **Key custody:** the `age` keypair is generated once at bootstrap, off the box. The public half is placed on the VPS (non-secret — it can only encrypt). The private half lives in the operator's password manager and a printed offline recovery copy per the runbook; it is **never** on the VPS, and is a wholly separate key from the credential keyring (§6.1/§6.4) — the two are never stored together, so a single compromised secret can't unwrap both live credentials and backup history. Rotation (new keypair) is a manual runbook step on suspected compromise; old backups stay readable under the old private key, which the runbook says to retain until its last covered backup expires.
- **RPO:** ≤ 15 minutes via continuous, individually-encrypted WAL segment shipping; the nightly full dump is the tested fallback restore point (RPO 24h) if WAL replay is unavailable.
- **RTO:** ≤ 4 hours for total loss of the VPS — provision a replacement box from the same IaC, decrypt the latest dump and WAL with the offline private key, restore, replay, redeploy the last tagged release image. ≤ 30 minutes for a bad-migration rollback that doesn't need a box rebuild (§8.1).
- **Restore drills:** monthly (SECURITY C-73, tightened from this section's earlier quarterly draft), scripted, and run **off the prod VPS** — the operator's own machine, or a throwaway scratch VPS spun up and destroyed for the drill — never a scratch compose project on the box itself: that would put the `age` private key on the VPS for the drill, which this section's own key-custody rule (§7.2 above) forbids outright, drill or not. Checked against a fixture query (trial balance sums to zero); logged as a runbook checklist item, not assumed to work from the backup job succeeding alone. The drill script also asserts the store credential cannot delete an existing object (C-73's second test, and the measurement the append-only bullet above is waiting on).

7.3 Sizing

CX32-class (4 vCPU / 8 GB RAM) rather than CX22 (2 vCPU / 4 GB): Postgres (prod + staging) + web + worker + Caddy fit comfortably inside 8 GB idle, and the extra headroom absorbs backup/WAL-shipping runs, migration bursts, and the staging Playwright smoke without contending with prod. **Unmeasured**

— confirmed against real resource metrics after POLY-M1 lands; downgrading to CX22 is a one-line IaC change if it proves oversized.

7.4 Observability & Operations

- **Logs:** web / worker emit structured JSON to stdout (one schema — `ts`, `level`, `req_id`, `entity_id?`, `event`, ...; exact fields agreed with backend-lead before POLY-M1 ships anything that logs). Docker's `json-file` driver with `max-size` / `max-file` rotation captures them; `docker compose logs` / `journalctl` is the query surface at this scale. No log-aggregation service until a second box or third-party hosting is in play.
- **Metrics:** a `/api/v1/health` route reports DB connectivity, last successful migration, and last successful backup timestamp. Polled by a free external uptime monitor. Container resource metrics via `docker stats` on demand; no Prometheus/Grafana until scale or a second operator justifies the extra service.
- **Alerts** (all by email — no on-call rotation for a household of one): backup job failure or a backup older than 26h; `/health` failing for >5 minutes; CI deploy-to-prod failure.
- **Dashboard:** `/health` plus `docker compose ps` is the v1 dashboard — the thing the team-lead can point to at launch. A Grafana-class dashboard is a documented future initiative once observability volume justifies the extra container.

7.5 Secrets Delivery & Postgres Config

DevOps's INFRA-level implementation of SECURITY §6.2 and §12.3's remaining asks.

- **Secrets as files, not env vars (SECURITY §6.2, C-74):** every operator secret lives under `/etc/polycarpic/secrets/` on the host (root:root, files `0400`, directory `0700`), mounted into containers as Docker Compose `secrets:` at `/run/secrets/<name>`. Apps read them through a `*_FILE` convention (e.g. `DATABASE_PASSWORD_FILE=/run/secrets/db_password`), never a bare env var — `docker inspect`, `/proc/<pid>/environ` and an error reporter dumping `process.env` all leak plain env vars, never a file only the right container can read. Per-container scoping: `web` never gets the credential private keyring or the backup private key; only `worker` mounts the keyring secret. Staging gets its own distinct secret values and no keyring at all, since it only ever runs the `fake` provider. No secret is ever baked into an image or passed as a build arg; the app refuses to start outside development if a required secret file is missing.
- **Postgres logging (SECURITY C-75):** `log_parameter_max_length = 0` and `log_parameter_max_length_on_error = 0` (bound parameter VALUES never reach the log, even on error — a sealed credential's ciphertext is one thing, but a query parameter carrying a raw token before it's sealed is exactly what this guards), `log_statement = 'ddl'` (schema changes only, never row data). Set in the Postgres compose service's config, asserted by a config-check in the DB battery (§8's DB battery row).

- **Connection-pooler mode, asserted on both targets:** the VPS's Postgres and Neon (the secondary target) must both run in a pooler mode where `set_config(..., true)` (the transaction-local RLS GUC, ARCH §3.3) is safe under connection reuse — transaction-mode pooling, not session-mode, on any pooler in front of either target (PgBouncer on the VPS if one is added later; Neon's own built-in pooler). A CI assertion connects through each target's pooler and confirms the GUC resets between transactions on a reused connection (the same pool-reuse property qa-engineer's pgTAP battery checks at the RLS layer, asserted here at the pooler-config layer instead).

7.6 Vercel Secondary Target: Two-Project Split (OQ-13)

Resolves OQ-13; SECURITY C-61/R-3 @ 3c66ce1 (phase/plan-security). DevOps builds the CI check; the split itself is a SecEng ruling.

- **Topology:** the Vercel target is two Vercel projects deployed from the same repo, not one. `web` exposes only user-facing routes, holds only public keys and the `poly_app` DB credentials. `jobs` exposes only cron routes (each checking `CRON_SECRET` with `timingSafeEqual`), holds the credential keyring and the `poly_worker` credentials. This is the same TB-4 (`web` seals / `worker` opens) split the VPS target already has via separate compose services — Vercel just can't isolate them inside one deployment, so it takes two deployments instead.
- **CI check, three legs (SECURITY C-61's own "build tests"):**
 - *Route manifest vs. allowlist.* Each project's build output (SvelteKit's route manifest under `.vercel/output/functions/`) is diffed against a checked-in allowlist per project — an exact-set match, not a subset check, so a route silently added to the wrong project's allowlist is caught too, not just one silently missing. `jobs`'s allowlist is exactly the cron routes; `web`'s is everything else.
 - *No keyring reference in the `web` build.* A bundle-scan (grep the built output for the keyring module's resolved import path, same mechanism the existing `packages/secrets` import-boundary lint uses source-side) asserts `open()` and the keyring module never got bundled into `web`'s build artifact — catches a route misclassified into `web`'s allowlist that would otherwise pass the manifest check by having the right PATH but the wrong CODE behind it.
 - *Node build 404s job routes.* The VPS's `adapter-node` build (one image, `web / worker / migrate` entrypoints, §8.1) is hit on its `web` entrypoint at each cron route path and must 404 — job routes exist only behind the `worker` entrypoint there, and this is the cross-check that the VPS and Vercel targets enforce the identical TB-4 boundary by two different mechanisms (separate compose service vs. separate Vercel project) without drifting apart.
- **Env check in CI (C-61):** a Vercel API call lists each project's configured env var *names* (never values) and asserts the keyring variable name is present in `jobs` and absent from `web` — catches an operator (or a future IaC change) setting it on the wrong project directly in the Vercel dashboard, which the build-time checks above can't see.
- **Cron frequency — measured:** Vercel's Hobby plan caps cron jobs at once per day, with timing guaranteed only within the hour (Pro lifts this to per-minute). Since Hobby is already the ceiling this

self-hoster fallback target targets, daily is not a choice DevOps is making for cost reasons — it is the only cadence Hobby allows at all. Import polling, EOD prices and period-end jobs on the Vercel target therefore run once daily, a real (accepted) data-freshness gap against the VPS target's more frequent worker loop; SecEng's framing ("daily polling is acceptable for this fallback target") holds given Vercel is explicitly the secondary, not the primary, deploy target (kickoff §2.8).

- **Cost:** two Hobby-tier Vercel projects, same as one — Hobby is free per project, so the split costs nothing beyond the CI check's own build time (one extra `adapter-vercel` build, ~same cost as the existing one). If POLY-77 ever needs Pro-tier cron frequency, that's a per-project upgrade, not a re-architecture.

8. CI/CD Pipeline

Owned by `devops-engineer` ; this is the shape, *INFRA* holds the detail.

Stage	Tool	Gate
Change detection	Turborepo affected graph; docs-only changes skip test jobs	—
Lint + types	ESLint (incl. layer-boundary rules), <code>svelte-check</code> , <code>tsc</code>	Must pass
Unit / package tests	Vitest, affected packages only	Must pass
DB battery	Apply all migrations to a fresh Postgres as <code>poly_migrator</code> ; pgTAP RLS + trigger battery (incl. inversion self-test, pool-reuse test)	Must pass when <code>packages/db</code> or any schema changes
OpenAPI freshness	Regenerate and diff against the committed spec	No diff
Build	<code>adapter-node</code> image; <code>adapter-vercel</code> build (both Vercel projects, §7.6)	Both must build
Vercel topology check	Route manifest vs. allowlist per project; no keyring reference in the <code>web</code> build; node build 404s job routes; env-name check via Vercel API (§7.6)	Must pass when <code>apps/web</code> routes or Vercel project config change
Smoke (integration milestones)	Playwright: login, import a batch, confirm a draft, view a statement	Must pass
Security scan	Dependency audit, secret scan	No high-severity findings
Deploy → staging	Pull-based: the box polls and verifies attestation (§8.1)	Auto on merge to <code>main</code>
Deploy → prod	Same mechanism, triggered by a release tag cut by <code>/merge-pr</code>	Tag only

8.1 Deploy mechanism

- **Decision, revised per SECURITY C-83: pull-based, not push-over-SSH.** An earlier draft of this section chose GitHub Actions → SSH push; SecEng's explicit preference is pull-based specifically

because it means **no GitHub-held credential can ever reach prod** — the box, not a GitHub Actions runner, decides when and what to deploy. A `systemd` timer on the box (every 5 minutes) runs a small poll script: query the GitHub Releases API for the newest `v*` tag (prod) or the registry for the newest `main`-branch image digest (staging); if it differs from what's running, pull the image **by digest** (never a mutable tag), then verify. A failed check at either verification step aborts the poll cycle and alerts (§7.4) — the box never runs an unverified image.

- **Attestation verification, pinned per SecEng's delta (SECURITY C-73/C-83 @ 61b4d2a):**
`gh attestation verify <digest> --repo richmosko/polycarpic --signer-workflow <path-to-the-release-workflow>` — pinning the repo and the exact signer workflow, not just "an attestation exists" (an unpinned check would accept a provenance attestation from any workflow in any repo the same GitHub identity can sign for). **Prod adds a second, independent check:** the `v*` tag itself must carry an SSH signature verifiable against a checked-in `allowed_signers` file (the same signing `/merge-pr` already does before cutting a release tag, SECURITY §8 "Signing") — so a compromised registry or attestation service alone still can't move prod without also forging a tag signature staging never checks.
- **The box's GitHub credential:** a fine-grained PAT, scoped to this one repo only, `contents: read` + `packages: read`, with an expiry (rotated per the runbook, same cadence discipline as §6.4's other operator secrets) — never a classic PAT, never org-wide. If the GHCR package and the repo are ever made public, the PAT is dropped entirely: an unauthenticated `gh api` / registry pull needs no credential on the box at all, which is strictly less attack surface than a scoped-but-present one.
- **Build once, promote:** CI builds a single `adapter-node` image per commit, pushes it to GHCR tagged with the git sha, with an SBOM (CycloneDX) and a provenance attestation attached (SECURITY §8); that same image digest is promoted from staging to prod on tag, never rebuilt. This rules out a staging/prod code-drift class of bug. `adapter-vercel` builds separately in CI as the secondary-target check (§7) but is not deployed by this pipeline. The one image serves three compose services through different entrypoints: `web`, `worker` and `migrate`. Each gets only its own secrets; the credential keyring is mounted into `worker` alone (TB-4), and lint keeps `open()` out of the web bundle.
- **Deploy script** (idempotent, same for staging/prod, invoked by the poll timer once it has a verified digest to move to): `docker compose pull` (by digest) → one-off `migrate` → `docker compose up -d` → poll `/health` with retry → non-zero exit on failure without swapping traffic (Caddy keeps routing to the last healthy containers until the new ones pass health).
- **Rollback (one button):** a runbook command points the poll script's "desired digest" file at the previous release's digest and re-runs the deploy script directly (no need to wait for the next timer tick, and no dependency on GitHub's Releases API being reachable at rollback time — the operator already knows the prior digest from the deploy log). Migrations are additive / backward-compatible by convention (expand-contract — no destructive migration ships in the same release as the code that depends on the drop), so a code rollback never needs a schema rollback. A schema-level rollback is rare and falls back to the DR restore path (§7.2).

- **Operator secrets** on the box live as files under `/run/secrets/`, one per container that needs it (§7.5) — placed once during VPS bootstrap and never touched by CI or the poll script, which only ever reads a public digest and a public attestation. Neither transmits a secret value onto, or reads one off, the box.

9. Integration Points

Limits marked (unmeasured) are to be verified against provider docs before each adapter's milestone.

Service	Purpose	Auth	Limits	Failure mode
Fake source	Deterministic transactions for three personas; fake EOD walk	none	none	n/a — used in dev, CI and staging
CSV / OFX file import	Credential-free import path (SecEng's preferred default for self-hosters)	none (user upload)	upload size cap	Parse errors surface per row; nothing is posted without confirmation
SimpleFIN Bridge	First real aggregator	Setup token → access URL (bearer-equivalent; sealed)	Limited daily requests per connection (unmeasured)	Job retries with backoff; the dashboard shows "last synced"; no data loss because de-dup is by hash
Plaid	Second aggregator	client id + secret (operator secret) + per-item access token (sealed)	Per-item pricing; rate limits per endpoint (unmeasured)	As above; item-login-required surfaces a relink prompt; <code>/item/remove</code> on unlink
Schwab / IBKR APIs	Positions, trades, lots (later)	OAuth 2.0 / broker-specific	unmeasured	As above
EOD price provider	Daily closes for held securities	API key (operator secret)	Provider-dependent (OQ-5)	Fall back to the last observation, flagged stale; source priority lets a better source replace it later
BLS API v2	CPI-U (CUURooooSAo), monthly	Free per-operator key (low-tier secret); v1 without a key	500 queries/day with a key (PRD §8)	Use the latest stored month; manual CSV import fallback

Service	Purpose	Auth	Limits	Failure mode
Resend	Transactional email (verification, magic link, MFA recovery)	API key (operator secret)	Free-tier volume cap (unmeasured)	Auth email fails loudly; OAuth login still works
Google / Apple / GitHub	OAuth identity providers	OAuth 2.0 client credentials per operator	n/a at this scale	That login method is unavailable; others still work
Off-box backup store	Encrypted backups (Hetzner Storage Box or S3-compatible)	Store credential (operator secret)	Storage quota	Backup job alerts; the RPO clock starts

Operator secrets (provider client secrets, API keys, the credential keyring, the DB passwords) live as files on the VPS, each mounted only into the container that needs it (the keyring into `worker` only; SECURITY §6.2). On the secondary target they are Vercel env vars. They never live in the repo or the database. **User credentials** (access URLs, access tokens) live sealed in the database (§3.1, SECURITY).

10. Trade-offs & Alternatives

- **Home-grown GL over pgledger.** We own correctness, and in exchange we get a model that fits (multi-leg, tenancy, multi-currency-ready lines). §6.1.
- **Drizzle over dbmate.** One TS schema for queries and migrations, plus BetterAuth in the same chain, at the cost of hand-written files for triggers and functions and an unproven policy differ. The spike gates the choice. §6.2.
- **Transaction-local RLS context over per-user DB roles or session SET .** Per-user roles don't scale with memberships and complicate pooling; session SET leaks across pooled connections. `set_config(..., true)` is pool-safe on both targets.
- **Import split into raw (L2) and posting (L4).** This honours "import is the lowest layer" for raw data while keeping dependencies strictly downward. It costs one extra package.
- **One entry per entity for cross-entity transfers.** Each entity's books balance on their own; there are no cross-entity entries. The cost is that matching must pair across entities and post twice (OQ-6).
- **Signed single amount per line over debit/credit columns.** The balance check becomes a single `SUM = 0` ; presentation derives Dr/Cr from sign and the account's normal side.
- **Balances computed, not stored.** No running-balance column until a measured need. That removes a whole class of lock contention and drift bugs, at the cost of a query that grows with history (indexed).
- **pg-boss (Postgres) over Redis for jobs.** No extra service. The cost is job throughput we will never need.
- **Web pages call packages directly, not through /api/v1 .** This skips a serialization hop and keeps SSR fast. The two paths share DTO types, so they can't drift; the cost is that the web UI doesn't dogfood the HTTP API (the Playwright smoke and the native client will).
- **Staging on the prod VPS.** Saves a second box. Staging holds fake data only, so a staging compromise exposes no real books, but the two share a kernel. INFRA may split them later.
- **Pull-based deploy over push-over-SSH (SECURITY C-83).** No credential GitHub holds can ever reach prod directly, at the cost of a systemd timer on the box and up to a five-minute delay between a tag landing and the box noticing it (acceptable for a single-operator deployment with no SLA).
- **Age-recipient backup encryption over restic's symmetric password (SECURITY §6.6).** A compromised box can encrypt new backups but never decrypt any backup, past or future — the cost is giving up restic's deduplication, negligible at this data volume.
- **Reports as data (declarative) over code.** Users can clone and edit, and upgrades don't clobber clones. The cost is building a small formula grammar. Scripting stays deferred (PRD §3).

11. Milestone Map

Development milestones under `POLY-V1`, ordered by the layer model. The files in `process/cairn/milestones/` are authoritative; this is the rationale.

Milestone	Layers	Stories	Integration check at close
POLY-M1 — Foundations	Lo, L1, L6 shell	POLY-76 (spike), POLY-62	A user signs in, creates an entity, grants a viewer; the RLS battery is green; migrate runs in CI and on staging
POLY-M2 — Ledger & Import	L2, L3, L4	POLY-64, POLY-65	The fake persona imports, drafts are confirmed into balanced immutable entries, and categories / tax tags apply
POLY-M3 — Transfers, Reconciliation & Dashboard	L4, L6	POLY-71, POLY-72, POLY-63	The couple persona's transfers net CIT to zero; the custodial reconciliation matches; the dashboard is under 2 s
POLY-M4 — Securities & Market Data	L2 refdata, L3 securities, L5 portfolio	POLY-67, POLY-69, POLY-75	Persona 3's lots, EOD valuation and allocation are correct; the real / nominal toggle works
POLY-M5 — Statements	L5 reports	POLY-73, POLY-68	Seeded templates render; clones survive a template upgrade
POLY-1.0 — Tax, Sharing & Real Data (GA)	L5 tax, L6, L2 real adapter	POLY-66, POLY-70, POLY-77 (real-data adapter)	Tags <code>v1.0.0</code> : the Principal runs one entity end to end on real data (PRD goal 1)

12. Open Questions

Each has an owner and a target resolution point.

#	Question	Owner	Resolve by
OQ-1	<i>Resolved, revised by SecEng review 70c9do5 (SECURITY §6.3–§6.5):</i> HPKE (X25519 / HKDF-SHA256 / AES-256-GCM). <code>web</code> seals; only <code>worker</code> holds the keyring and opens. Stored as <code>enc</code> , <code>ciphertext</code> , <code>key_id</code> , <code>alg_version</code> . <code>pgcrypto</code> and KMS are rejected. Rotation is a re-seal job.	seceng	Closed 2026-09-28
OQ-2	<i>Resolved (SECURITY §12, §4.2.1, §4.3):</i> the §3.3 shape is confirmed, with additions (a)–(l) folded in. Roles are uniform across entity types.	seceng + qa-engineer	Closed 2026-09-28
OQ-3	<i>Resolved (SECURITY §4.4):</i> jobs run under the linking user's membership (<code>linked_by_user_id</code> NOT NULL), with a <code>principal_invalid</code> state that fails loudly. The system policy clause was rejected.	seceng	Closed 2026-09-28
OQ-4	<i>Resolved, revised by 70c9do5 (SECURITY §4.5):</i> the purge function is owned by the NOLOGIN <code>poly_purge</code> role, with role-targeted DELETE policies. Eight guards, including export freshness and a zero-rows-remain assert. See §3.4.	seceng	Closed 2026-09-28
OQ-5	Which EOD price provider (free tier, coverage of mutual funds / ETFs)? mosko-fintech used provider-implied prices plus FMP.	architect	Start of POLY-M4
OQ-6	Cross-entity transfers (e.g. Person → Household joint account): what counterpart account does each side book — contribution / distribution, or a due-to / due-from pair?	Principal (via team-lead)	Start of POLY-M3
OQ-7	Real-data path for GA: SimpleFIN adapter, CSV/OFX file adapter, or both? PRD goal 1 needs one. Tracked as POLY-77 in POLY-1.0; its scope waits on this answer.	Principal (via team-lead)	Before POLY-M2 closes
OQ-8	Should intermediate milestones cut <code>0.x</code> releases so prod can run on real data before GA, or does everything before GA live on staging with fake data?	Principal (via team-lead)	Before POLY-M1 closes

#	Question	Owner	Resolve by
OQ-9	Resolved — see §7.2: staging co-located on the prod VPS; backup store is Hetzner Storage Box (S3-compatible as a config-only alternative); RPO ≤ 15 min (WAL) / 24h (nightly dump fallback), RTO ≤ 4h (full loss) / ≤ 30 min (bad-migration rollback).	devops-engineer	INFRA v1 — done 2026-09-28
OQ-10	Drizzle spike outcome (§6.2), which confirms or flips the migration-tool decision.	architect + backend-lead	POLY-76, first issue of POLY-M1
OQ-11	<i>Resolved in outline (§2.4)</i> : the six DESIGN screens are mapped to package DTOs. Field-level DTO shapes get pinned in each package's design file at the start of its milestone.	ux-designer + architect	Per milestone
OQ-12	Prior-year filed tax per entity per jurisdiction (safe-harbor input): a manual entry per year, or derived from the ledger's settled filing entries?	architect + Principal	Start of POLY-1.0
OQ-13	Resolved — see §7.6: the Vercel target is two projects from one repo (SECURITY C-61), <code>web</code> holding public keys + <code>poly_app</code> only, <code>jobs</code> holding the keyring + <code>poly_worker</code> . CI enforces the split (route manifest, bundle scan, node-build 404, env-name check). Hobby-plan cron is capped at once daily — measured, accepted as this fallback target's data-freshness trade-off.	seceng + devops-engineer	Before POLY-77 — done 2026-09-28

Source hand-offs folded in: PRD-import spillover (kickoff §2.4–§2.11 architecture detail) and SecEng's Research-phase notes. Their [Plan] items surface here as OQ-1 through OQ-4, and in §3.3 / §3.4.